# Software Design Pattern Specification

## Chain of Responsibility Pattern in PEC Clearance Automation Engine

**Institution:** Punjab Engineering College (Deemed to be University), Sector 12, Chandigarh - 160012
**Implementation:** `backend/app/patterns/chain_of_responsibility.py`

---

### 1. Pattern Overview & Intent

The **Chain of Responsibility (CoR)** is a behavioral design pattern that allows passing requests along a chain of handlers. Upon receiving a request, each handler decides either to process the request or to pass it along the chain.

At **Punjab Engineering College (PEC)**, clearance requests must satisfy strict institutional policies:

1. Multi-departmental sign-offs must occur across distinct autonomous bodies:
   - Dr. B.R. Ambedkar Central Library
   - PEC Hostels (Shivalik / Kurukshetra / Himalaya / Aravali / KCH)
   - Gymkhana & Sports Complex
   - Departmental Research Labs (CSE / ECE / Mechanical / Electrical / Aerospace)
   - Accounts & Finance Branch (Caution Money Reconciliation)
2. Certain approvals can proceed **concurrently / in parallel** without blocking one another (Gate 1).
3. Critical checkpoints have **sequential prerequisites** (e.g., Department Labs requires all Gate 1 clearances; Final Accounts requires all previous gates before refunding caution money).
4. Each clearance event requires **cryptographic HMAC signing and audit logging**.

---

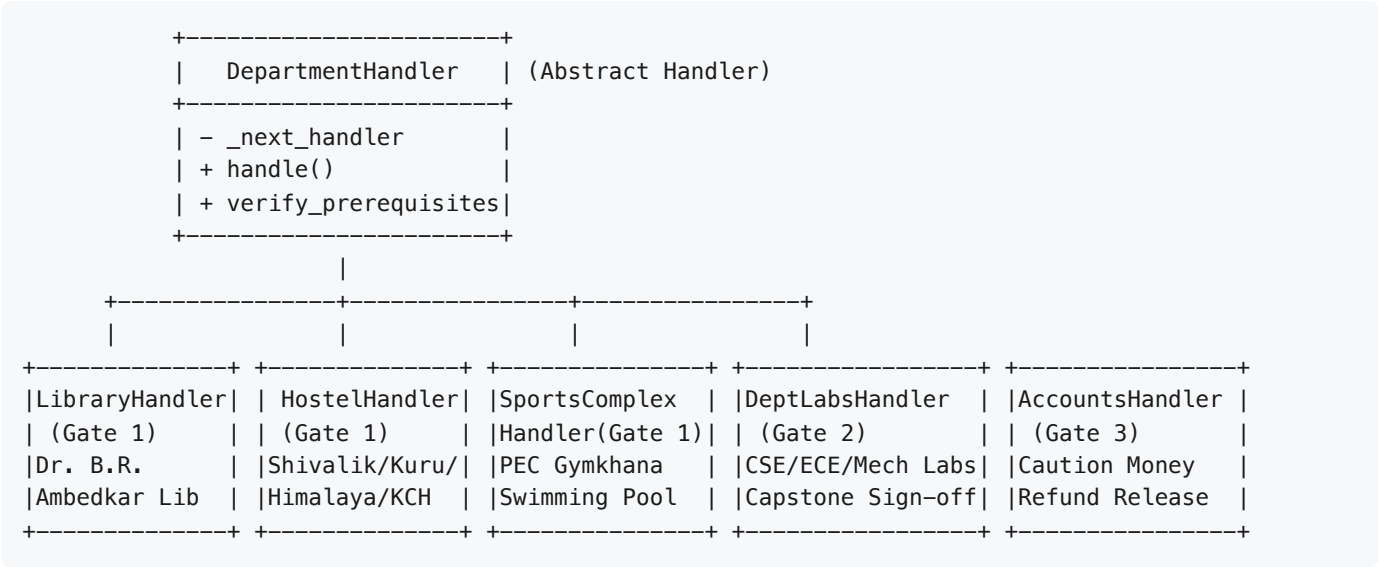
### 2. Architectural Comparison: Why Chain of Responsibility?

| Architectural Approach | Extensibility | Coupling | Dependency Handling | Academic Suitability |
|---|---|---|---|---|
| **Hardcoded Monolithic** `if/else` | Very Low (modifying order breaks codebase) | High coupling to all PEC departments | Fragile nested conditionals | Poor (Code Smell: Long Method) |
| **State Pattern** | Medium | Medium | Difficult to model 3 parallel states simultaneously | Fair |
| **Chain of Responsibility (Adopted)** | **Very High** (new facilities plugged in as new handlers) | **Loose coupling** (handlers adhere to uniform interface) | **Elegant prerequisite gates & delegation** | **Exemplary Software Engineering design** |

---

## 3. Concrete Implementation Details at PEC

### 3.1 Handler Hierarchy




### 3.2 Handling Parallel Gates within the PEC Chain

In traditional textbook CoR, every link is strictly serial. In institutional clearance, students can visit the Library, Hostels, and Gymkhana in any order.

Our architecture addresses this via:

- **Gate 1 Parallel Handlers:** `LibraryHandler`, `HostelHandler`, and `SportsComplexHandler` report `verify_prerequisites() == True` at any time. A student can have their Hostel approved first, Gymkhana second, and Library third.
- **Gate 2 Dependency Gate:** When `DepartmentLabsHandler` is invoked, its `verify_prerequisites()` executes a dependency resolution check:

  ```
  gate1_codes = ["LIBRARY", "HOSTEL", "SPORTS"]
  for code in gate1_codes:
      if dept_statuses.get(code) != ApprovalStatusEnum.APPROVED:
          return False, f"Prerequisite unmet: '{code}' clearance must be APPROVED before Departmer
  ```

- **Gate 3 Final Financial Gate:** `AccountsHandler` verifies that both Gate 1 and Gate 2 are `APPROVED` before releasing caution money and issuing the final institutional sign-off.

### 3.3 Dynamic State Refresh & Digital Signature Mints

When any handler successfully signs off on a request:

1. An HMAC-SHA256 signature is calculated:

   ```
   Digital Signature = HMAC-SHA256(Secret_Key, staff_id + dept_code + request_id + timestamp)
   ```

2. `refresh_pipeline_gates()` evaluates downstream approvals:
   - Changes downstream status from `WAITING_PREREQUISITE` to `PENDING` when unblocked.

- If all 5 departments are approved, transitions `ClearanceRequest.overall_status` to `APPROVED` and generates a certificate digest (e.g. `PEC-CLR-YYYY-XXXXX`).